1	2	3	$\sum$

Freie Universität Berlin

Algorithmen, Datenstrukturen und Datenabstraktion (WiSe 18/19)

Docent: Prof. Dr. Wolfgang Mulzer

Tutorin: Valeria Zahoransky; Tutorial: 07

Exercise 03

Release: 29.10.2018; Due: 09.11.2018

Viktoriya Kraleva, Samuel Enderwitz

---

1 O-Notation (__/10)

(a) *Sort the following functions in increasing order. (If $f(n) \in O(g(n))$, but not $g(n) \in O(f(n))$, then $f(n)$ should come before $g(n)$.) Justify your answer.*

(1) n^3 *(2)* $\log_2 n$ *(3)* $1{,}8^n$ *(4)* n *(5)* 3^n *(6)* $\sqrt{n}$ *(7)* $n(\log_2 n)^2$ *(8)* n^2

$$\log_2 n \leq \sqrt{n} \leq n \leq n(\log_2 n)^2 \leq n^2 \leq n^3 \leq 1,8^n \leq 3^n$$

Begründung: $\log_2 n \leq n$
Also muss dann: $\frac{\log_2 n}{n} \leq c$ Wir wissen,dass $n \geq 0$,weil es den Anzahl der Elementen entspricht.
$log(n) < n$
$n - \log_2(n) = n - \frac{\log(n)}{\log 2} > 0$
(Wegen erste Ableitung) ist dies $h(x) = 1 - (\frac{1}{n}) * 0.301$ (0.301) ist logarithmuis 10 von 2 (aproximiert) - also beschreibt genau der Wachstum der Funktion
Wir betrachten,was sich bei $x \in (0, 1)$ passiert:
$\log_2(n)$ wird < 0 sein und deswegen auch:
$n - \log_2(n) > 0$
Wenn $n > 1$ wird sich die Funktion immer wachsen.(offensichtlich auch von der ersten Ableitungsfunktion)
Wenn $n = 1$ wird die $log(n) = 0$ sein und damit auch $1 - 0 > 0$
Damit haben wir die Gleichung für $\log_2(n) < n$ bewiesen. Also : $\lim_{x\to\infty} \frac{\log_2 n}{n} = 0 \leq 1$,also ist dann unser $c = 1$.
Begründung: $\log_2 n \leq \sqrt{n}$
Wir wissen jetzt,dass $\log_2 n \leq n$
Damit können wir n mit $\sqrt{n}$ ersetzen.
und damit gilt :
$\frac{1}{2} \cdot \log_2 n \leq \sqrt{n}$
$\log_2 n \leq 2 \cdot \sqrt{n}$
Hier müssen wir die Definition für die obere Schranke von $f(x)$ anwenden und zwar:
$\forall c, c \in \mathbb{R}, c > 0 : \exists n_0, n_0 \in \mathbb{N} : \forall n, n \in \mathbb{N}, n \geq n_0 : f(n) \leq c.g(n)$
und hier ist zwar c = 2 und damit ist $2 \cdot \sqrt{n}$ eine obere Schranke für $\log_2 n$ und $\log_2 n \in O(\sqrt{n})$

Aber umgekehrt gibt es keine solche konstante,also $g(n) \notin O(\log_2 n)$.
Begründung: $\sqrt{n} \leq n$
Also wir quadrieren die ganze Ungleichung:
$n \leq n^2 * c$ und danach teilen wir durch n (wir dürfen das machen,weil $n \in \mathbb{N}$ Annahme $n > 0$
$1 \leq n * c$
Dann ist unsere Konstante $c = 1$ und damit auch ist $\sqrt{n} \in O(n)$. Umgekehrt ist es aber nicht, weil es muss : $\lim_{x\to\infty} \frac{1}{n} \geq 0$ sein, und dies ist ab $n > 1$ nicht erfüllt. Also $n \notin O(\sqrt{n})$
Begründung: $n \leq n(\log_2 n)^2$
Dies folgt sofort,wenn wir die Ungleichung durch n teilen:
$1 \leq (\log_2 n)^2 * c$
$\lim_{x\to\infty} \frac{1}{(\log_2 n)^2} \leq c$- offensichtlich warum.Die grenzwert ist 0 und bei dem Wert $c = 1$ ist die Ungleichung erfüllt.Dann ist unser $c = 1$ und damit auch $n \in O(n(\log_2 n)^2)$
Umgekehrt kann es aber nie sein,weil:
$\lim_{x\to\infty} \frac{1}{(\log_2 n)^2} \geq c$ und ist nur dann erfüllt, wenn $c < 0$,aber die Konstante muss aber positiv sein,also im Allgemein nicht wahr.$n(\log_2 n)^2 \notin O(n)$ **Begründung:** $n(\log_2 n)^2 < n^2$
$\lim_{x\to\infty} \frac{n(\log_2 n)^2}{n^2} \leq c$
$\lim_{x\to\infty} \frac{(\log_2 n)^2}{n} \leq c$
Aus dem Grund,dass wir schon vorher bewiesen haben,dass $\log_2 n$ langsamer wächst als n können wir Behaupten,dass der Grenzwert dieser Funktion : $\lim_{x\to\infty} \frac{(\log_2 n)^2}{n} = \lim_{x\to\infty} \frac{(\log_2 n)}{\sqrt{n}} = 0 \leq \sqrt{c}$,also $c = 1$ und damit ist die Bedingung erfüllt,weil wir schon bewiesen haben,dass $\sqrt{n}$ schneller als $\log_2 n$ wächst. Umgekehrt geht es nicht,weil $\lim_{x\to\infty} \frac{(\sqrt{n})}{log_2 n} = \infty$ ist.
Begründung: $n^2 \leq n^3$
Muss dann:
$\lim_{x\to\infty} \frac{n^2}{n^3} = 0 \geq c$, also ist dann c in unserem Fall $c = 1$. Damit ist $O(n^2) \in O(n^3)$
Aber wenn man dies über $\lim_{x\to\infty} \frac{n^3}{n^2} = \infty < c$ beweisen muss,geht es nicht,deshalb $O(n^3) \notin O(n^2)$.
Begründung: $n^3 \leq 1.8^n$
Wir machen nochmal den selben Trick und zwar :
$\lim_{x\to\infty} \frac{n^3}{1.8^n} = 0$,weil man kann ja auch von der Graph sehen,dass ab 14 $1.8^n > n^3$ ist.
Damit würde ich die Konstante als $c = 1.8^14 = 3784$ machen und auf dieser Weise wird die immer $\frac{n^3}{1.8^n} < c$ erfüllt sein und aus diesem Grund auch $O(n^3) \in O(1.8^n)$.
Umgekehrt geht es nicht,weil ab $1.8^{14} = g$ wächst die Funktion ziemlich schneller als $f = n^3$,also kann es keine solche feste Konstante c existieren $O(1.8^n) \notin O(n^3)$.
Begründung: $1.8^n \leq 2^n$
Aus dem Grund,dass nur der Exponent hier von Bedeutung ist, weil die Potenzen die gleichen sind, glaube ich,dass es ziemlich offensichtlich ist warum die Funktion $O(1.8^n) \in O(2^n)$ und $c = 1$. Umgekehrt geht es nicht, auch offensichtlich warum.

(b) *Assume that a program has running time $f(n)$, where $f(n)$ is one of the functions in (a). Assume further that in ten years computers will be 1000 times faster than today. Determine the size of the problems that we will be able to solve in ten years in the*

same time that we need today in order to solve a problem of size $n = 20$.

Wir nehmen an,dass : $f_i(20) = c_{20}$
Dann muss : $\frac{1}{1000} f_i(n) = c_{20}$
Damit wählen wir die drei Komplexitätsklassen von a):
1. **O(n)** $f(n) = n + c_1 \in O(n)$
$f(20) = 20$
$f_i(n) = 1000 \cdot 20 = 20000 \rightarrow n = 20$
2. **O(n^2)** $f(n) = a \cdot n^2 + a_1 \cdot n + c_1 \in O(n) \wedge a > 0$
$f(20) = n^2 = 400$
$f_i(n) = 1000 \cdot 400 = 20000 \Rightarrow n^2 = 40000 \Rightarrow n = 200$
2. **O(n^3)** $f(n) = a \cdot n^3 + a_1 \cdot n^2 + a_1 \cdot n + c_1 \in O(n) \wedge a > 0$
$f(20) = n^3 = 8000$
$f_i(n) = 1000 \cdot 8000 = 800000 \Rightarrow n^3 = 800000 \Rightarrow n = 20 * \sqrt[3]{100}$

2 Simultaneous Minimum and Maximum (__/10)

(a) *Let A be an array of n pairwise comparable elements. Give a simple and efficient algorithm to find the minimum and the maximum of A. Determine exactly the number of comparisons that are needed in the worst case (as a function of n).*

Annahme:
Wir haben eine RAM mit einem Befehlssatz genommen, den wir in Assembler gelernt haben. Wir haben dabei Register r1 bis r5. Im 1. Speichersegment steht die Länge des Arrays und danach kommen die Einträge des Arrays.
In beiden Fällen (Minimum, Maximum) müssen wir den Array bis zum Ende durchgehen.
Am Ende speichern wir das Minimum in das 1. Speichersegment.

Zusätzlich nehmen wir an, dass das Array absteigend sortiert ist und das Minium somit am Ende des Arrays steht. Dadurch haben wir ein Maximum an Operationen und somit den worst case.

Beim Minimum/Maximum haben wir einen worst case, wenn das Array absteigend/aufsteigend sortiert ist und das Minium/Maximum am Ende/Anfang des Arrays steht. Damit haben wir eine Maximale Anzahl an Operationen. Da wir aber n in Abhängigkeit der Vergleichen setzen, ist dies irrelevant.

```
MOV r1, [0] % size of array
MOV r2, 1 % current pos
MOV r3, [r2] % the actual minimum
_loop:
    INC r2 % increase current pos
    CMP r2, r1 % compare current pos with size of array
    JG _end % if at end of array finish
    r4, [r2] % element at current pos
    CMP r4, r3 % compare current pos with minimum
    JGE _loop % if current pos is greater that minimum -> skip
    r3 = r5 % if new minium write
_end:
```

```
    MOV [0], r3 % write the minimum
    RET
```

Listing 1: Minium

```
MOV r1, [0] % size of array
MOV r2, 1 % current pos
MOV r3, [r2] % the actual maximum
_loop:
    INC r2 % increase current pos
    CMP r2, r1 % compare current pos with size of array
    JG _end % if at end of array finish
    r4, [r2] % element at current pos
    CMP r4, r3 % compare current pos with maximum
    JLE _loop % if current pos is greater that maximum -> skip
    r3 = r5 % if new minium write
_end:
    MOV [0], r3 % write the maximum
    RET
```

Listing 2: Maximum

Da die RAM bei beiden sehr ähnlich ist und alle wesentlichen Operationen gleich sind, können wir eine Analyse für beide durchführen. In jedem Durchlauf vergleichen wir unsere Position mit der größe des Arrays. Dabei gibt es noch einen Vergleich mehr, wenn wir eigentlich schon fertig sind.
Für jedes Element in der Liste vergleichen wir den Wert an der aktuellen Position mit unserem Minimum. Da wir aber das erste Element am Anfang raus nehmen und wir dann erst mit dem zweiten Elemen Vergleichen haben wir dort einen Vergleich weniger.

Somit haben wir eine Funktion in von: $f(n) = (n+1) + (n-1) = 2n$

(b) *Solve the same problem using a divide and conquer approach, by dividing A into two parts if $n \geq 2$. Determine exactly the number of comparisons that are needed in the worst case (as a function of n). You may assume that n is a power of two.*

Es gelten die gleichen Annahmen wie bei (a).
Dazu nehmen wir an, dass bei dem Aufruf der Minimum Funktion der Speicher sich selbständig so verändert, das die Funktionen ihn benutzen können. Alternativ dazu müsse man die Minimum- und Maximum-Funktion erweitern und eine Startposition übergeben.
Ausdrücke in runden Klammern sind Funktionsaufrufe.

```
MOV r1, [0] % size of array
CMP r1, 2 % compare size with 2
JL _one % is size < 2
MOV r2, (minimum(0:n/2)) % minimum from first part of list
MOV r3, (minimum(n/2:n)) % minimum from second part of list
CMP r2, r3
JLE _first % if minimum of first is lower
MOV [0], r3 % write second as minimum
RET
_first:
    MOV [0], r2 % write first as minimum
```

```
    RET
_one:
    MOV r1, (minimum) % call minimum with whole list
    RET
```

Listing 3: Divide Minium

```
MOV r1, [0] % size of array
CMP r1, 2 % compare size with 2
JL _one % is size < 2
MOV r2, (maximum(0:n/2)) % maximum from first part of list
MOV r3, (maximum(n/2:n)) % maximum from second part of list
CMP r2, r3
JGE _first % if maximum of first is lower
MOV [0], r3 % write second as maximum
RET
_first:
    MOV [0], r2 % write first as maximum
    RET
_one:
    MOV r1, (maximum) % call maximum with whole list
    RET
```

Listing 4: Divide Maximum

Wir vergleichen für jede Liste die Größe mit 2 und dann die zwei Minima/Maxima miteinander. Somit haben wir dort 2 Vergleiche.
Da wir die Listen durch zwei Teilen und die Teillisten parrallel nach Minima/Maxima suchen lassen haben wir nun $2 \cdot \frac{n}{2} = n$.
Somit haben wir mit "Teile und Hersche" $n + 2$ Vergleiche.

3 Simple Data Structures in Java (__/10)

(a) *Consider the following interface for a Stack in Java.*

```
public interface Stack<E> {
  void push(E element);
  E pop();
  E top();
  int size();
  boolean isEmpty();
}
```

Extend the interface with reasonable exccleptions. Then, implement a class ***ListStack<E>*** *and a class* ***DynamicArrayStack<E>****. These two classes should implement* ***Stack****, using a linked list and a dynamic array. Take care of appropriate error handling and exceptions and test your program in different inputs.*

```
package de.smuu.add.e03;

import java.awt.*;
import java.io.File;
import java.io.IOException;
```

```java

public class Main {

    public static void main(String[] args) {
        System.out.println("Test for DynamicArrayStack:");
        DynamicArrayStack<String> stringDynamicArrayStack = new
    DynamicArrayStack<>(1);
        stringDynamicArrayStack.push("Hallo");
        System.out.println(stringDynamicArrayStack.top());
        // size of array is to small and create new one
        stringDynamicArrayStack.push("Hola");
        System.out.println(stringDynamicArrayStack.size());
        System.out.println(stringDynamicArrayStack.pop());
        System.out.println(stringDynamicArrayStack.pop());
        System.out.println(stringDynamicArrayStack.size());
        System.out.println();

        System.out.println("Test for ListStack:");
        ListStack<String> stringListStack = new ListStack<>();
        stringListStack.push("Tschuss");
        System.out.println(stringListStack.top());
        stringListStack.push("Adios");
        System.out.println(stringListStack.size());
        System.out.println(stringListStack.pop());
        System.out.println(stringListStack.pop());
        System.out.println(stringListStack.size());
        System.out.println();
    }
}
```

Listing 5: Main

```java
package de.smuu.add.e03;

import java.util.EmptyStackException;

@SuppressWarnings("unchecked")
public class ListStack<E> implements Stack {

    private ListNode<E> head;

    public ListStack () {
        this.head = null;
    }

    @Override
    public void push(Object element) {
        head = new ListNode(element, head);
    }

    @Override
    public E pop() throws EmptyStackException {
        if (isEmpty()) {
            throw new EmptyStackException();
        }
        E obj = head.getElement();
        head = head.getNext();
```

```java
        return obj;
    }

    @Override
    public E top() throws EmptyStackException {
        if (isEmpty()) {
            throw new EmptyStackException();
        }
        return head.getElement();
    }

    @Override
    public int size() {
        ListNode<E> act = head;
        int i = 0;
        while (act != null) {
            act = act.getNext();
            i++;
        }
        return i;
    }

    @Override
    public boolean isEmpty() {
        return head == null;
    }
}
```

Listing 6: ListStack

```java
package de.smuu.add.e03;

import java.util.EmptyStackException;

@SuppressWarnings("unchecked")
public class DynamicArrayStack<E> implements Stack {

    private E[] array;
    private int pos;

    public DynamicArrayStack() {
        this(10);
    }

    public DynamicArrayStack(int size) {
        if (size <= 0) {
            throw new IllegalArgumentException("Size of array must be >= 1");
        }
        this.array = (E[]) new Object[size];
        pos = 0;
    }

    @Override
    public void push(Object element) {
        if (pos == array.length) {
            resize();
```

```java
        }
        array[pos] = (E) element;
        pos++;
    }

    @Override
    public E pop() throws EmptyStackException {
        if (isEmpty()) {
            throw new EmptyStackException();
        } else {
            E obj = array[pos-1];
            pos--;
            return obj;
        }
    }

    @Override
    public E top() throws EmptyStackException {
        if (isEmpty()) {
            throw new EmptyStackException();
        } else {
            return array[pos-1];
        }
    }

    @Override
    public int size() {
        return pos;
    }

    @Override
    public boolean isEmpty() {
        return size() == 0;
    }

    private void resize() {
        if (pos == array.length) {
            E[] newArray = (E[]) new Object[array.length * 2];
            for (int i = 0; i < array.length; i++) {
                newArray[i] = array[i];
            }
            //Alternativ (recommended from IntelliJ):
            //System.arraycopy(array, 0, newArray, 0, array.length);
            array = newArray;
        }
    }
}
```

Listing 7: DynamicArrayStack

(b) Write a program that determines for a given string of parentheses whether it is structured correctly. We allow parentheses of type (), [], <> and {}. Use an appropriate data structure. For example "(A){L[P]3}" is structured correctly, but "(A){L[P}3]" and "(A){L[P3" are not.

Hint: You can access individual characters in a Java String using the method `charAt`.

```java
package de.smuu.add.e03;

import java.util.ArrayList;
import java.util.List;

public class CheckBrackets {

    private static List<Character> openBrackets = new ArrayList<>();
    private static List<Character> closeBrackets = new ArrayList<>();

    static {
        // add all open and close brackets to static lists
        openBrackets.add('(');
        closeBrackets.add(')');
        openBrackets.add('[');
        closeBrackets.add(']');
        openBrackets.add('<');
        closeBrackets.add('>');
        openBrackets.add('{');
        closeBrackets.add('}');
    }

    public static boolean check(String string) {
        // you can use both of them
        //DynamicArrayStack<Character> brackets = new DynamicArrayStack
    <>();
        Stack<Character> brackets = new ListStack<>();
        for (int i = 0; i < string.length(); i++) {
            char c = string.charAt(i);
            if (openBrackets.contains(c)) {
                brackets.push(c);
            } else if (closeBrackets.contains(c)){
                int pos = closeBrackets.indexOf(c);
                if (brackets.pop() != openBrackets.get(pos)) {
                    return false;
                }
            }
        }
        return brackets.size() == 0;
    }
}
```

Listing 8: CheckBrackets

(c) (voluntary, 5 extra points) *Find out about* unit testing *and* JUnit. *Obtain the JUnit-framework and implement 5 respective test cases for the two Stack implementation in (a) and 2 test cases for your solution to (b).*

Mit Unit tests kann man Klassen und Methoden gut testen. JUnit ist dabei ein Framework für Java.
Ein JUnit-Test kann entweder gelingern oder misslingen. Dabei muss man zwischen Fehler und falsches Ergebnis unterscheiden. Ein Fehler tritt eher unerwartet auf und zeigt einen Fehler im Code auf. Ein falsches Ergebnis kann gewollt sein, da man eine Funktion/Klasse auf besondere Eingaben testen möchte.

```java
package de.smuu.add.e03;

import org.junit.runner.JUnitCore;
import org.junit.runner.Result;
import org.junit.runner.notification.Failure;

import java.io.FileWriter;
import java.io.IOException;
public class TestRunner {

    //TODO: write better version of file writing!
    private static FileWriter fileWriter;

    static {
        try {
            fileWriter = new FileWriter("testOutput.txt");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }



    public static void main(String[] args) {
        print("Test Stack:");
        testStack();
        print("");
        print("Test CheckBrackets:");
        testBrackets();

        try {
            fileWriter.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    private static void testStack() {
        Result result = JUnitCore.runClasses(TestStack.class);

        for (Failure failure : result.getFailures()) {
            print(failure.toString());
        }

        int amount = result.getRunCount();
        int failed = result.getFailureCount();
        int ignored = result.getIgnoreCount();
        int succeed = amount-failed-ignored;

        print("Run " + amount + " tests");

        print(succeed + " succeed, " + failed + " failed and " + ignored
    + " ignored");
    }

    private static void testBrackets() {
        Result result = JUnitCore.runClasses(TestBrackets.class);
```

```java

        for (Failure failure : result.getFailures()) {
            print(failure.toString());
        }


        int succeed = result.getRunCount();
        int failed = result.getFailureCount();
        int ignored = result.getIgnoreCount();

        int amount = succeed + failed + ignored;

        print("Run " + amount + " tests");

        print(succeed + " succeed, " + failed + " failed and " + ignored
    + " ignored");
    }

    private static void print(String message) {
        System.out.println(message);
        try {
            fileWriter.write(message + "\n");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Listing 9: TestRunner

```java
package de.smuu.add.e03;

import junit.framework.Assert;
import org.junit.Test;

public class TestStack {

    @Test
    public void testListStack0() {
        ListStack<String> listStack = new ListStack<>();
        Assert.assertEquals(listStack.size(), 0);
    }

    @Test
    public void testListStack1() {
        ListStack<String> listStack = new ListStack<>();
        listStack.push("Hii");
        Assert.assertEquals(listStack.size(), 1);
    }

    @Test
    public void testListStack2() {
        ListStack<String> listStack = new ListStack<>();
        listStack.push("Hii");
        Assert.assertFalse("shoud not be empty", listStack.isEmpty());
    }

```

```java
    @Test
    public void testListStack3() {
        ListStack<String> listStack = new ListStack<>();
        listStack.push("Hii");
        Assert.assertNotNull("shoud not be null", listStack.top());
    }

    @Test
    public void testListStack4() {
        ListStack<String> listStack = new ListStack<>();
        listStack.push("Hii");
        Assert.assertNotNull("shoud not be null", listStack.pop());
    }


    @Test
    public void testDynamicArrayStack0() {
        DynamicArrayStack<String> dynamicArrayStack = new DynamicArrayStack<>();
        Assert.assertEquals(dynamicArrayStack.size(), 0);
    }

    @Test
    public void testDynamicArrayStack1() {
        DynamicArrayStack<String> dynamicArrayStack = new DynamicArrayStack<>();
        dynamicArrayStack.push("Hii");
        Assert.assertEquals(dynamicArrayStack.size(), 1);
    }

    @Test
    public void testDynamicArrayStack2() {
        DynamicArrayStack<String> dynamicArrayStack = new DynamicArrayStack<>();
        dynamicArrayStack.push("Hii");
        Assert.assertFalse("shoud not be empty", dynamicArrayStack.isEmpty());
    }

    @Test
    public void testDynamicArrayStack3() {
        DynamicArrayStack<String> dynamicArrayStack = new DynamicArrayStack<>();
        dynamicArrayStack.push("Hii");
        Assert.assertNotNull("shoud not be null", dynamicArrayStack.top());
    }

    @Test
    public void testDynamicArrayStack4() {
        DynamicArrayStack<String> dynamicArrayStack = new DynamicArrayStack<>();
        dynamicArrayStack.push("Hii");
        Assert.assertNotNull("shoud not be null", dynamicArrayStack.pop());
    }
}
```

Listing 10: TestStack

```java
package de.smuu.add.e03;

import junit.framework.Assert;
import org.junit.Test;

public class TestBrackets {


    @Test
    public void testCheckBrackets() {
        String message = "([<([])>{}(<{}>)])";
        Assert.assertTrue("should be true", CheckBrackets.check(message)
    );
    }

    @Test
    public void testCheckBrackets1() {
        String message = "([<([])>{}(<{}>)])(<)";
        Assert.assertTrue("should be false", CheckBrackets.check(message
    ));
    }
}
```

Listing 11: TestBrackets

```
Test Stack:
Run 10 tests
10 succeed, 0 failed and 0 ignored

Test CheckBrackets:
testCheckBrackets1(de.smuu.add.e03.TestBrackets): should be false
Run 3 tests
2 succeed, 1 failed and 0 ignored
```

Listing 12: TestOutput